ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА
Факультет електроніки та комп'ютерних технологій

ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №8

«Мережеве та асинхронне програмування. REST API»

з курсу «Професійний Python»
Варіант №13 — «Система обліку комп'ютерної техніки»

Виконав: Капарник Дмитро Андрійович

Група: ФЕП-32

Перевірив: доцент Куньо Іван Михайлович

Дата: 18.10.2026

Львів — 2026

1. Мета роботи

Створити REST API з валідацією й persistence та дослідити асинхронні HTTP-запити з керованим паралелізмом.

2. Індивідуальний варіант і постановка задачі

Варіант №13 — система обліку комп'ютерної техніки. Реалізувати ресурси devices/manufacturers, GET/POST/PATCH/DELETE, пошук, фільтри й статистику; додати HTTPX, Task/gather, timeout, retry та Semaphore.

Наскрізний проєкт продовжує ЛР №1. Базові поля: інвентарний номер, тип, виробник, модель і вартість. Збережено пошук, фільтр виробника, середню/загальну вартість і визначення найдорожчого пристрою.

3. Структура проєкту та створені модулі

```
inventory-service/
  pyproject.toml, README.md, .gitignore
  src/computer_inventory/
    api.py
    async_client.py
    database.py
    models.py
  src/inventory_service/  # фінальний пакет запуску
  tests/
  scripts/
  docs/GITHUB.md
```

4. Контракт REST API

Метод	Шлях	Успіх / призначення
GET	/devices	200: пошук, фільтр, offset/limit
POST	/devices	201: реєстрація
GET	/devices/{device_id}	200: один пристрій
PATCH	/devices/{device_id}	200: нова вартість
DELETE	/devices/{device_id}	204 без тіла
GET	/manufacturers	200: виробники
POST	/manufacturers	201: виробник
GET	/manufacturers/{manufacturer_id}/devices	200: техніка виробника
GET	/devices/statistics, /statistics	200: підсумки
GET	/manufacturers/details	200: асинхронна агрегація провайдера

Метод	Шлях	Успіх / призначення
GET	/health	200: стан застосунку та БД

Path parameters: device_id та manufacturer_id. Query parameters: manufacturer, inventory_number, offset >= 0 і 1 <= limit <= 100. Невідомий id повертає 404, дублікат — 409, порушення схеми — 422, недоступний провайдер — 502. Статичний /devices/statistics зареєстровано перед динамічним маршрутом id.

5. Pydantic та перевірка даних

```
class DeviceCreate(BaseModel):
    inventory_number: str = Field(min_length=1, max_length=100)
    type: str = Field(min_length=1, max_length=100)
    manufacturer: str = Field(min_length=1, max_length=100)
    model: str = Field(min_length=1, max_length=200)
    value: Decimal = Field(ge=0, le=1000000000000, decimal_places=2)
    location: str | None = Field(default=None, max_length=200)

    @field_validator("inventory_number", "type", "manufacturer", "model")
    @classmethod
    def nonempty(cls, value: str) -> str:
        if not value.strip():
            raise ValueError("Field cannot be blank")
        return value.strip()

class DevicePatch(BaseModel):
    value: Decimal = Field(ge=0, le=1000000000000, decimal_places=2)

class DeviceView(BaseModel):
    id: int
    inventory_number: str
    type: str
    manufacturer: str
    manufacturer_id: int
    model: str
    value: str
    location: str | None

class ManufacturerCreate(BaseModel):
    name: str = Field(min_length=1, max_length=100)

    @field_validator("name")
    @classmethod
    def nonempty(cls, value: str) -> str:
        if not value.strip():
            raise ValueError("Manufacturer cannot be blank")
        return value.strip()
```

Порожні або лише пробільні текстові поля відхиляються. Вартість обмежена $0…10^{12}$ грн та двома десятковими знаками; Decimal не допускає NaN/Infinity за цією схемою. JSON-відповідь містить грошове значення рядком.

6. Реалізація обробників та persistence

```
def session_dependency() -> Iterator[Session]:
    with Session(engine) as session:
        yield session

@app.get("/devices", response_model=list[DeviceView])
def list_devices(
    database: Database,
    manufacturer: str | None = None,
    inventory_number: str | None = None,
    offset: int = Query(0, ge=0),
    limit: int = Query(50, ge=1, le=100),
) -> list[DeviceView]:
    repository = DeviceRepository(database)
    if inventory_number:
        found = repository.find(inventory_number)
        rows = [found] if found is not None else []
        if manufacturer:
            rows = [
                row
                for row in rows
                if row.manufacturer.name.casefold()
                == manufacturer.strip().casefold()
            ]
    else:
        rows = repository.all(manufacturer)
    return [view(row) for row in rows[offset : offset + limit]]

@app.post("/devices", status_code=201, response_model=DeviceView)
def add_device(payload: DeviceCreate, database: Database) -> DeviceView:
    device = Device(
        payload.inventory_number,
        payload.type,
        payload.manufacturer,
        payload.model,
        payload.value,
    )
    try:
        row = DeviceRepository(database).add(device, payload.location)
        result = view(row)
        database.commit()
        return result
    except IntegrityError as error:
        database.rollback()
        raise HTTPException(409, "Inventory number already exists") from error

@app.get("/devices/{device_id}", response_model=DeviceView)
def get_device(device_id: int, database: Database) -> DeviceView:
    row = DeviceRepository(database).get(device_id)
    if row is None:
        raise HTTPException(404, "Device not found")
    return view(row)

@app.patch("/devices/{device_id}", response_model=DeviceView)
def update_device(
    device_id: int, payload: DevicePatch, database: Database
```

```
    ) -> DeviceView:
        try:
            row = DeviceRepository(database).change_cost(device_id, payload.value)
            result = view(row)
            database.commit()
            return result
        except LookupError as error:
            raise HTTPException(404, "Device not found") from error

    @app.delete("/devices/{device_id}", status_code=204)
    def delete_device(device_id: int, database: Database) -> Response:
        try:
            DeviceRepository(database).delete(device_id)
            database.commit()
            return Response(status_code=204)
        except LookupError as error:
            raise HTTPException(404, "Device not found") from error

    @app.get(
        "/manufacturers/{manufacturer_id}/devices", response_model=list[DeviceView]
    )
    def manufacturer_devices(
        manufacturer_id: int, database: Database
    ) -> list[DeviceView]:
        row = database.get(ManufacturerRow, manufacturer_id)
        if row is None:
            raise HTTPException(404, "Manufacturer not found")
        return [view(device) for device in DeviceRepository(database).all(row.name)]
```

Сесія створюється dependency-функцією й закривається після запиту. Синхронний SQLite ORM використовується у звичайних def-обробниках FastAPI, щоб не виконувати блокувальний SQL у власному async-коді. Точний пошук номера йде через індекс repository.find; фільтрація й статистика використовують persistence layer ЛР7.

7. Асинхронний провайдер

```
async def fetch_manufacturers(
    client: httpx.AsyncClient,
    names: list[str],
    concurrency: int = 3,
    retries: int = 2,
    timeout: float = 2.0,
) -> list[dict[str, Any]]:
    if not 1 <= concurrency <= 20 or len(names) > 100 or retries < 0 or timeout <= 0:
        raise ValueError("Invalid async client settings")
    semaphore = asyncio.Semaphore(concurrency)

    async def fetch(name: str) -> dict[str, Any]:
        for attempt in range(retries + 1):
            try:
                async with semaphore, asyncio.timeout(timeout):
                    response = await client.get(
                        "/manufacturers/" + quote(name, safe="")
                    )
```

```
                    response.raise_for_status()
                    result = response.json()
                    if not isinstance(result, dict):
                        raise ValueError("Provider response must be an object")
                    return result
            except (httpx.TransportError, TimeoutError, httpx.HTTPStatusError) as error:
                retryable = (
                    not isinstance(error, httpx.HTTPStatusError)
                    or error.response.status_code >= 500
                    or error.response.status_code == 429
                )
                if attempt == retries or not retryable:
                    raise
                await asyncio.sleep(0.01 * 2**attempt)
        raise RuntimeError("Unreachable retry state")

    tasks = [asyncio.create_task(fetch(name)) for name in names]
    try:
        return await asyncio.gather(*tasks)
    finally:
        for task in tasks:
            if not task.done():
                task.cancel()
        await asyncio.gather(*tasks, return_exceptions=True)


    @app.get("/manufacturers/details")
    async def manufacturer_details(
        names: list[str] = Query(..., min_length=1, max_length=100),
    ) -> list[dict[str, Any]]:
        base_url = os.getenv("PROVIDER_URL", "http://127.0.0.1:8001")
        try:
            async with httpx.AsyncClient(base_url=base_url) as client:
                return await fetch_manufacturers(client, names)
        except (httpx.HTTPError, TimeoutError) as error:
            raise HTTPException(502, "Manufacturer provider unavailable") from error
```

Task/gather збирають результати у вхідному порядку. Semaphore обмежує активні запити; timeout накладає часову межу. Повторюються транспортні помилки, timeout, HTTP 429 та 5xx з коротким експоненційним backoff; решта HTTP-помилок передається одразу. Після невдачі незавершені задачі скасовуються та очікуються, щоб не залишити фонову роботу. Обсяг одного запиту обмежено 100 назвами.

8. Приклади запиту й фактичної відповіді

```
POST /devices
{
  "inventory_number": "INV-001",
  "type": "Ноутбук",
  "manufacturer": "Lenovo",
  "model": "ThinkPad E14",
  "value": "32000"
}
```

```
HTTP 201
{
  "id": 1,
  "inventory_number": "INV-001",
  "type": "Ноутбук",
  "manufacturer": "Lenovo",
  "manufacturer_id": 1,
  "model": "ThinkPad E14",
  "value": "32000.00",
  "location": null
}
```

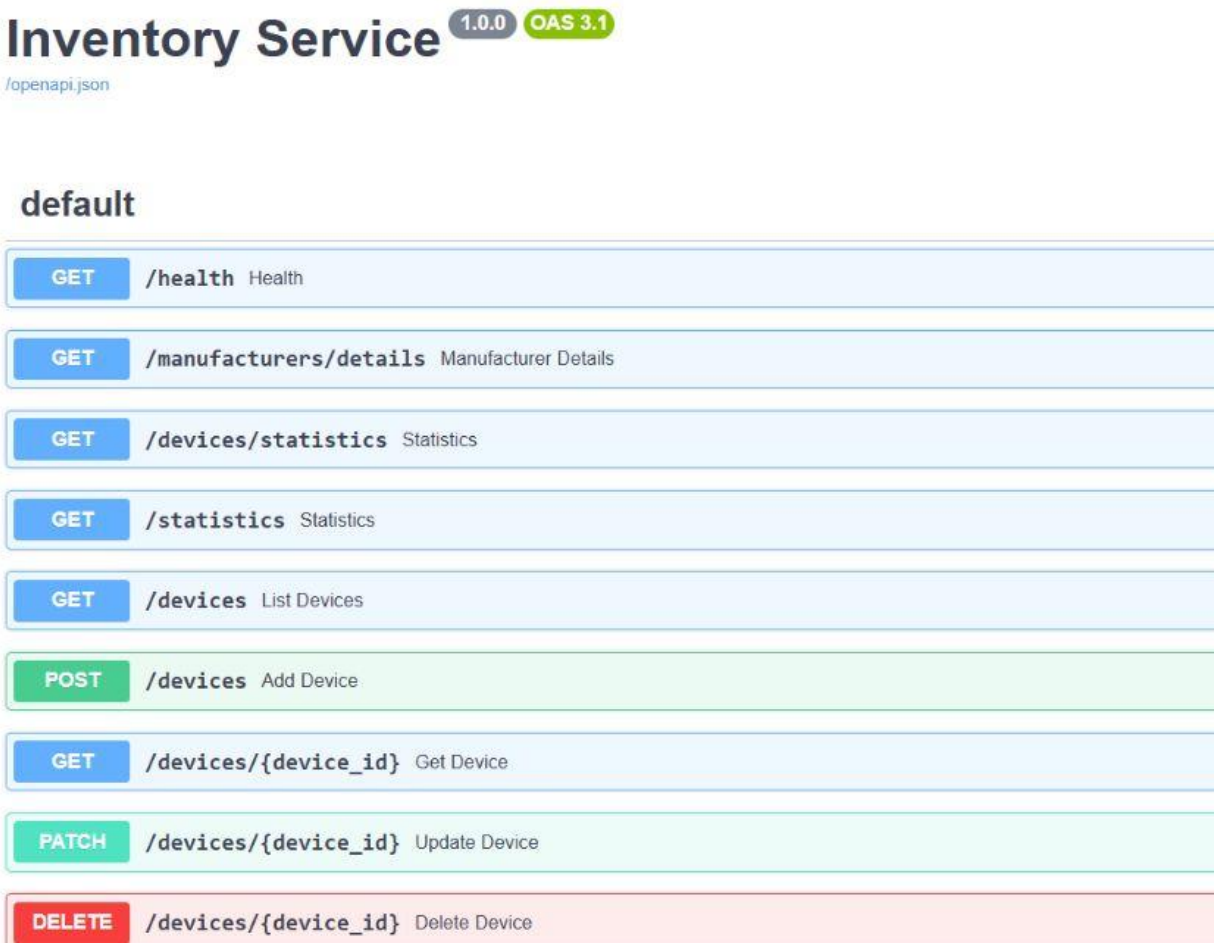


Рисунок 1 — Swagger UI фактично запущеного Inventory Service

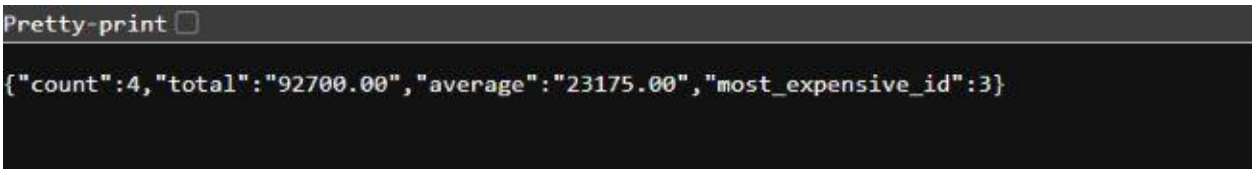


Рисунок 2 — Фактична відповідь GET /devices/statistics (обрізано порожню область екрана)

9. API tests і перевірка асинхронності

```
def test_api_crud_statistics(client, payload):
    response = client.post("/devices", json=payload)
    assert response.status_code == 201
    body = response.json()
    device_id = body["id"]
    assert body["value"] == "120.50"
    assert client.get(f"/devices/{device_id}").status_code == 200
    assert len(client.get("/devices", params={"manufacturer": "Dell"}).json()) == 1
    assert len(client.get("/devices", params={"inventory_number": "A-1"}).json()) == 1
    assert (
        client.get(
            "/devices", params={"inventory_number": "a-1", "manufacturer": "Dell"}
        ).json()[0]["id"]
        == device_id
    )
    assert client.get("/devices", params={"inventory_number": "missing"}).json() == []
    assert client.get("/statistics").json()["total"] == "120.50"
    assert client.get("/devices/statistics").status_code == 200
    mid = body["manufacturer_id"]
    assert len(client.get(f"/manufacturers/{mid}/devices").json()) == 1
    assert (
        client.patch(f"/devices/{device_id}", json={"value": "200"}).json()["value"]
        == "200.00"
    )
    assert client.post("/devices", json=payload).status_code == 409
    assert client.delete(f"/devices/{device_id}").status_code == 204
    assert client.get(f"/devices/{device_id}").status_code == 404
    assert client.get("/statistics").json()["count"] == 0
    assert client.get("/health").json() == {"status": "ok"}

def test_async_retry_concurrency():
    attempts = {}
    active = 0
    maximum = 0

    async def handler(request):
        nonlocal active, maximum
        active += 1
        maximum = max(maximum, active)
        await asyncio.sleep(0.001)
        active -= 1
        name = request.url.path.split("/")[-1]
        attempts[name] = attempts.get(name, 0) + 1
        return httpx.Response(503 if attempts[name] == 1 else 200, json={"name": name})

    async def run():
        async with httpx.AsyncClient(
            transport=httpx.MockTransport(handler), base_url="https://provider.example"
        ) as client:
            return await fetch_manufacturers(
                client, ["Dell", "HP", "Lenovo"], concurrency=2
            )

    result = asyncio.run(run())
    assert len(result) == 3 and maximum <= 2
```

```
    assert attempts == {"Dell": 2, "HP": 2, "Lenovo": 2}

def test_async_failure_timeout():
    async def run():
        async def handler(request):
            await asyncio.sleep(0.1)
            return httpx.Response(200, json={})

        async with httpx.AsyncClient(
            transport=httpx.MockTransport(handler), base_url="https://provider.example"
        ) as client:
            with pytest.raises(TimeoutError):
                await fetch_manufacturers(client, ["Dell"], retries=0, timeout=0.001)
            with pytest.raises(ValueError):
                await fetch_manufacturers(client, ["Dell"], concurrency=0)

    asyncio.run(run())
```

Тести використовують окрему БД та TestClient; async-провайдер — HTTPX MockTransport. Перша відповідь 503 примушує retry, друга 200 завершує запит; виміряна одночасність не перевищує 2. Окрема перевірка моделює timeout. Це відтворюваний HTTP-контракт без залежності від реального зовнішнього сервісу.

```
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.1.1, pluggy-1.7.0 --
C:\Users\user\Documents\Codex\2026-10-18\new-chat-2\outputs\computer-
inventory\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\user\Documents\Codex\2026-10-18\new-chat-2\outputs\computer-inventory
configfile: pyproject.toml
plugins: anyio-4.15.1, cov-7.1.0
collecting ... collected 14 items

tests/test_database.py::test_crud_filter_aggregates PASSED               [  7%]
tests/test_database.py::test_unique_batch_rollback PASSED                [ 14%]
tests/test_database.py::test_persistent_commit_dbapi PASSED              [ 21%]
tests/test_database.py::test_migration_and_foreign_key PASSED            [ 28%]
tests/test_database.py::test_empty_and_missing PASSED                    [ 35%]
tests/test_api.py::test_api_crud_statistics PASSED                       [ 42%]
tests/test_api.py::test_api_invalid_value[-1] PASSED                     [ 50%]
tests/test_api.py::test_api_invalid_value[NaN] PASSED                    [ 57%]
tests/test_api.py::test_api_invalid_value[1.001] PASSED                  [ 64%]
tests/test_api.py::test_api_invalid_value[1000000000001] PASSED          [ 71%]
tests/test_api.py::test_api_missing_and_manufacturers PASSED             [ 78%]
tests/test_api.py::test_async_retry_concurrency PASSED                   [ 85%]
tests/test_api.py::test_async_failure_timeout PASSED                     [ 92%]
tests/test_api.py::test_async_endpoint PASSED                            [100%]

============================== warnings summary ===============================
.venv\Lib\site-packages\fastapi\testclient.py:1
  C:\Users\user\Documents\Codex\2026-10-18\new-chat-2\outputs\computer-
inventory\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning:
Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa
```

```
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
======================== 14 passed, 1 warning in 1.92s ========================
```

Додатково живому Uvicorn на 127.0.0.1:8765 надіслано чотири POST (201), GET /health, /devices, /devices/statistics та /manufacturers/2/devices (усі 200). Отримано 4 пристрої, 92700.00 грн загальної та 23175.00 грн середньої вартості. У фінальному suite також перевірено async endpoint і його відповідь 502.

Код і історія лабораторних: https://github.com/dimakaparnyk/professional-python-labs. Для відповідного етапу використовується тег lab-08, фінальна інтегрована реалізація — main (перевірений CI commit 2e3fb3d).

10. Відповіді на контрольні запитання

1. Що таке комп'ютерна мережа?

Система взаємопов’язаних пристроїв, які обмінюються даними за протоколами.

2. Що таке клієнт-серверна архітектура?

Клієнт надсилає запити, а сервер надає ресурси та виконує операції.

3. Що таке HTTP?

Протокол прикладного рівня для обміну запитами та відповідями.

4. Що містить HTTP request?

Метод, адресу ресурсу, заголовки та, за потреби, тіло.

5. Що містить HTTP response?

Код стану, заголовки та, за потреби, тіло результату.

6. Для чого використовується GET?

Для отримання ресурсу без навмисної зміни його стану.

7. Для чого використовується POST?

Для створення ресурсу або виконання визначеної сервером операції.

8. Для чого використовується PUT?

Для повної заміни представлення ресурсу; ідемпотентність залежить від дотримання контракту.

9. Для чого використовується PATCH?

Для часткової зміни ресурсу.

10. Для чого використовується DELETE?

Для видалення зазначеного ресурсу.

11. Що означає status code 200?

Запит успішно виконано з представленням результату.

12. Що означає 201?

Ресурс успішно створено.

13. Що означає 204?

Операція успішна, тіло відповіді відсутнє.

14. Що означає 404?

Запитаний ресурс не знайдено.

15. Що означає 422?

Дані запиту не пройшли перевірку схеми або обмежень.

16. Що таке JSON?

Текстовий формат структурованих даних для обміну між клієнтом і сервером.

17. Що таке REST?

Архітектурний стиль роботи з ресурсами через узгоджений інтерфейс і незалежні запити.

18. Що таке resource у REST?

Іменована сутність або колекція, наприклад devices чи конкретний пристрій.

19. Що таке endpoint?

Поєднання HTTP-методу й шляху, яке реалізує визначену операцію.

20. Що таке path parameter?

Частина шляху, наприклад device_id у /devices/3.

21. Що таке query parameter?

Параметр після знака ?, наприклад manufacturer=Dell.

22. Що таке request body?

Передані клієнтом дані операції, зазвичай JSON для створення або зміни.

23. Що таке FastAPI?

Python-фреймворк побудови API з анотаціями типів, валідацією та OpenAPI.

24. Для чого використовується Uvicorn?

ASGI-сервер, який приймає HTTP-запити й запускає FastAPI-застосунок.

25. Для чого потрібний Swagger UI?

Для інтерактивного перегляду контракту API та виконання пробних запитів.

26. Що таке Pydantic?

Бібліотека схем і валідації Python-даних з перетворенням у визначені типи.

27. Що таке validation?

Перевірка структури, типів, діапазонів та інших обмежень вхідних даних.

28. Що таке response model?

Схема вихідних даних, що описує й перевіряє формат відповіді.

29. Що таке synchronous programming?

Виконання, за якого потік очікує завершення поточної операції перед продовженням.

30. Що таке asynchronous programming?

Кооперативне виконання кількох операцій, що можуть віддавати керування під час очікування.

31. Для чого використовується async?

Для оголошення корутинної функції або асинхронних контекстів та ітерації.

32. Для чого використовується await?

Для очікування awaitable з передаванням керування event loop.

33. Що таке coroutine?

Об’єкт виконання async-функції, який може призупинятися на await.

34. Що таке event loop?

Цикл, що планує корутини й обробляє готовність операцій введення та виведення.

35. Що таке Task?

Заплановане виконання корутини під керуванням event loop.

36. Для чого використовується asyncio.create_task()?

Для запуску корутини як незалежно запланованої Task.

37. Для чого використовується asyncio.gather()?

Для очікування кількох awaitable й отримання результатів у вхідному порядку.

38. Що таке timeout?

Обмеження допустимого часу очікування операції.

39. Для чого потрібний retry?

Для повтору тимчасово невдалої операції за обмеженою політикою.

40. Що таке backoff?

Затримка між повторами, часто зі збільшенням після кожної невдачі.

41. Для чого використовується Semaphore?

Для обмеження кількості одночасних активних операцій.

42. Що таке HTTPX?

HTTP-клієнт Python із синхронним та асинхронним інтерфейсами.

43. Чим httpx.AsyncClient відрізняється від синхронного клієнта?

AsyncClient дозволяє await запитів і не блокує event loop під час мережевого очікування.

44. Чому AsyncClient бажано повторно використовувати?

Він повторно використовує з’єднання та зменшує витрати створення клієнтів.

45. Навіщо обмежувати паралелізм?

Щоб не перевантажувати сервер, пул з’єднань, пам’ять або власну систему.

46. Чому не слід запускати тисячі tasks одночасно?

Кожна Task має витрати; обмежений пул або черга краще контролює великі набори.

47. Як REST API взаємодіє з Repository Layer?

Обробник перевіряє запит, викликає репозиторій і формує відповідь, не вбудовуючи SQL у контракт API.

48. Як FastAPI працює з Pydantic?

Використовує моделі для розбору запиту, перевірки параметрів, серіалізації та документації.

49. Як протестувати REST API?

Через TestClient або HTTPX з ASGITransport, окремою тестовою БД і перевірками станів та JSON.

50. Чим API integration test відрізняється від unit test?

Перевіряє маршрутизацію, валідацію, серіалізацію та persistence разом; unit test охоплює меншу частину.

11. Висновок

Реалізовано REST API над SQLite, схеми валідації, коректні HTTP-статуси й окремий асинхронний HTTP-клієнт. API tests та живі HTTP-запити підтвердили предметні операції варіанта 13.